

Aquí no se trata de “qué” hace una app, sino de “cómo” fue concebida. Cloud-native describe aplicaciones diseñadas desde su origen para entornos de nube: se construyen con microservicios, contenedores y APIs declarativas, se despliegan de forma automatizada y escalan horizontalmente con resiliencia intrínseca. Cloud-enabled (o lift-and-shift) nombra a los sistemas pensados para servidores físicos/VMs que luego se migran casi “tal cual” a la nube, con mínimas o nulas modificaciones arquitectónicas. La definición operativa de cloud-native que resume la CNCF enfatiza microservicios, infra inmutable, observabilidad y automatización para lograr sistemas desacoplados y tolerantes a fallos. Por contraste, AWS define rehost como mover una app a la nube sin cambiarla, y ubica replatform/refactor como etapas posteriores de modernización progresiva.

Spotify es un ejemplo paradigmático de cloud-native en producción a escala planetaria. En 2016 decidió dejar de operar centros de datos propios y migrar a Google Cloud Platform (GCP), en dos frentes: services (servicios online) y data (plataformas de datos). Ya entonces ejecutaba “multitud de microservicios pequeños”, y la migración adoptó servicios gestionados de GCP (Compute Engine, Cloud Storage, Pub/Sub, BigQuery, Dataflow) para acelerar y estabilizar operaciones. Google documenta que el esfuerzo movió 1,200 servicios online y flujos de datos (DAGs) con 20,000 jobs diarios afectando a más de 100 equipos, un indicador nítido del grado de descomposición y autonomía por dominio que caracteriza a lo cloud-native. Además, Spotify ha descrito públicamente que usa Google Cloud Storage para almacenar y servir audio, aprovechando la cercanía de cómputo para transcodificación y el peering global de red.

La filosofía cloud-native no es solo “dónde corre”, sino “cómo evoluciona”. Un ejemplo: la migración del Event Delivery de Kafka autogestionado a Pub/Sub y Dataflow gestionados, con el objetivo explícito de descargar operación a la plataforma y enfocarse en entregar valor. La CNCF, además, resume bien las ventajas: despliegues más rápidos, interrupciones mínimas por el diseño en microservicios y pago por uso más eficiente.

Una app cloud-enabled suele re-hospedarse en VMs y seguir desplegando de forma manual o con ventanas de downtime; escala verticalmente (bigger VM) y mantiene puntos únicos de fallo. Es una estrategia válida para reducir CAPEX y ganar experiencia, pero no captura los beneficios de agilidad, resiliencia y elasticidad fina que emergen cuando la arquitectura misma se reimagina para la nube. AWS sistematiza este viaje en las “7R”: del rehost inicial, pasando por replatform, hasta refactor para explotar de lleno el modelo cloud.

Ser cloud-native no es una bala de plata. La transición conlleva retos organizacionales (gobernanza, observabilidad distribuida, data ownership, SRE practices). Literatura reciente en ACM Queue subraya dificultades para adoptar y sostener microservicios (desde el diseño inicial hasta su operación a lo largo del tiempo), lo que exige disciplina de producto y plataforma, no solo tecnología. A nivel de principios, IEEE Computer Society destaca que los pilares de una arquitectura cloud-native robusta son escalabilidad, resiliencia, automatización y observabilidad; si no se invierte en ellos, el “microservicio” de nombre no entrega su promesa.

Cloud-enabled es a menudo el primer paso pragmático para salir del on-prem. Cloud-native es el destino cuando el negocio exige ciclos de cambio veloces, experiencias siempre disponibles y eficiencia operativa a escala. Spotify muestra cómo se ve ese destino en la

práctica: cientos de equipos desplegando miles de servicios sobre una plataforma gestionada, con developer experience como ventaja competitiva. Si tu app real hoy es monolítica, el camino recomendado no es el salto acrobático directo, sino una ruta guiada (rehost → replatform → refactor), priorizando dominios de alto impacto, adoptando servicios gestionados y creando una plataforma interna que haga lo correcto el camino de menor fricción.

Referencias:

Cloud Native Computing Foundation. (2023, September 4). Cloud-based versus cloud native: what's the difference?

<https://www.cncf.io/blog/2023/09/04/cloud-based-versus-cloud-native-whats-the-difference/>

Cloud Native Computing Foundation. (n.d.). Who we are (Cloud Native definition).

<https://www.cncf.io/about/who-we-are/>

Google Cloud. (2016, October 20). Introducing Coldline and a unified platform for data storage.

<https://cloud.google.com/blog/products/gcp/introducing-coldline-and-a-unified-platform-for-data-storage>

IEEE Computer Society. (2022, November 9). Cloud Native Architecture: 6 cloud native principles to know.

<https://www.computer.org/publications/tech-news/trends/cloud-native-architecture-principles-to-know/>